

Original Software Publication

UMLBot: Generative AI for converting natural language and code excerpts to editable UML diagrams

Ryan C. Godwin^{a,b,*}, Ryan L. Melvin^a

^a University of Alabama at Birmingham, Department of Anesthesiology and Perioperative Medicine, USA

^b University of Alabama at Birmingham, Department of Radiology, USA

ARTICLE INFO

Keywords:

UML
Generative AI
Software documentation
Software architecture
Reproducibility

ABSTRACT

Unified Modeling Language (UML) provides a standardized visual representation that facilitates communication and comprehension of design ideas, particularly for software systems with complex architectures and behaviors. However, creating and maintaining UML diagrams is time-consuming and often requires knowledge of specialized syntax. We present UMLBot, an open-source web application that uses large language models to generate editable PlantUML source and rendered UML diagrams from natural-language descriptions or directly from code excerpts. UMLBot supports iterative refinement through a chat-based interface, allowing users to request changes, inspect the generated PlantUML, and regenerate diagrams as requirements evolve. The application includes configurable OpenAI-compatible LLM access, PlantUML rendering, self-correction and retry with flexible deployment through Docker Compose.

Metadata

Metadata.

Nr	Code metadata description	Please fill in this column
C1	Current code version	v0.2
C2	Permanent link to code/ repository used for this code version	https://github.com/UABGH-Emergin-g-Technologies/UMLBot/releases/tag/v0.2
C3	Permanent link to reproducible capsule	https://codeocean.com/capsule/5503057/tree
C4	Legal code license	GPLv3
C5	Code versioning system used	git
C6	Software code languages, tools and services used	Python, TypeScript, Shell, JavaScript, Makefile, CSS, Dockerfile
C7	Compilation requirements, operating environments and dependencies	Docker, VSCode
C8	If available, link to developer documentation/manual	https://uabgh-emerging-technologies.github.io/UMLBot/
C9	Support email for questions	ryangodwin@uabmc.edu

1. Motivation and significance

Unified Modeling Language (UML[®]) is a standardized language for communicating engineering design intent and useful for Model Driven Engineering (MDE) [1]. UML provides utility across many different domains including healthcare [2–5], finance [6–8], and aerospace [9, 10]. Its success is partially due to UML serving as a cognitive and communicative aid for stakeholders at various levels (e.g., management, developer, IT, etc.) UML and its diagrammatic representations of the language identify components such as, actors, interfaces, and interactions, to name a few. These diagrams, while historically time consuming to build and maintain, are highly effective for complex engineering environments and support communication and traceability through the lifecycle of a project [11].

UML delivers its main benefits when it supports shared understanding and communication in a team and helps teams avoid expensive rework by validating design decisions early [12]. In many projects, the limiting factor is not necessarily drawing a diagram, but the friction of keeping UML design models useful over time. A recurring hurdle is the effort required to keep models and code synchronized, along with tooling that is hard to integrate, which discourages teams from treating UML as a dynamic artifact.

* Corresponding author.

E-mail address: ryangodwin@uabmc.edu (R.C. Godwin).

¹ - <https://www.omg.org/uml/>

If the time and effort of creating and updating UML drops substantially, teams can afford to keep diagrams current and revisit them as the design evolves, which makes UML more valuable as a durable communication and validation surface rather than a one-off sketch [12]. Here, we introduce UMLBot, an open-source, chat-based tool that generates and revises UML diagrams from a natural-language descriptions. The system uses a configured OpenAI-compatible large language model (LLM) endpoint to produce or revise a text-based UML specification in PlantUML², which is then rendered through the application. Users can request changes in plain language and iteratively refine both the diagram via chat or by directly modifying the underlying PlantUML source.

By turning conversational descriptions or code excerpts into version-controllable, editable PlantUML files, UMLBot is designed to reduce the effort required to create first-draft documentation and help teams maintain design artifacts as development progresses. The intent is not to outsource the reasoning behind architectural decisions, but to remove the friction of producing a usable first draft and then incrementally improving it and keeping it synchronized with source code.

Users interact through a web application that presents as inputs a diagram selection and chat panel alongside the outputs of generated PlantUML and the automatically rendered diagram. A typical session begins with selection of a diagram type (e.g., class or sequence), followed by a plain-language description of the desired design (optionally including code excerpts as context). The system generates PlantUML, renders the diagram, and updates the display after each user message.

The idea of using LLMs with UML is not itself novel. others have published on creating certain diagrams like activity diagrams [13] and use case diagrams [14], as well as the use of LLMs with UML modeling in educational settings [15]. Other studies have begun to assess the validity of LLM-generated diagrams, with results showing substantial variability across models, prompts, and diagram types [16]. These findings motivate workflows that make LLM-generated UML easier to inspect, revise, and maintain rather than treating generated diagrams as final artifacts.

UMLBot addresses this practical need by providing an open-source, locally deployable, web-based workflow centered on editable PlantUML. The contribution is not a new UML-generation algorithm, but the integration of LLM-assisted drafting, deterministic rendering, exposed source, error feedback rendering, and chat-based refinement into a single workflow. Prior studies have applied UML to document complex biomedical and clinical situations [17,18], and prior research has explored how textual requirements can be transformed into structured representations. UMLBot targets a complementary day-to-day development need: helping users produce reviewable UML drafts, revise them as requirements change, and preserve the resulting diagrams as version-controllable text artifacts.

2. Software description

UMLBot is implemented as a locally deployable web application for generating, rendering, and revising UML diagrams. The current release includes a Python/Gradio backend, a TypeScript/Next.js frontend, Docker Compose deployment support, and integration with a PlantUML rendering service. The web application coordinates user input, prompt construction, LLM-based PlantUML generation, rendering of the resulting diagram, and display of both the generated PlantUML source and rendered visual output. The backend supports the web application and serves as a standalone backend for other adaptations including agentic frameworks. The core workflow comprises (i) a chat and user-interface layer that captures user requests and displays both PlantUML and rendered diagrams; (ii) an orchestration layer that maintains conversational context and assembles prompts; (iii) a configurable LLM adapter that generates or revises PlantUML; (iv) a validation and error-handling layer that captures LLM, prompt, PlantUML-rendering, and network

errors and can optionally request an LLM-based repair; and (v) a PlantUML renderer that converts text specifications into standard diagram formats. As the intermediate artifact is plain text, diagrams can be stored, reviewed, and regenerated as part of a code repository.

2.1. Software architecture

Application development began from an LLM-based template architecture adapted from prior work [19]. The primary elements of UMLBot include:

- **Web application interface:** Users interact with UMLBot through a browser-based interface that supports free-text input, diagram-type selection, chat-based revision, and display of both raw PlantUML code and the corresponding rendered diagram. The repository provides a Gradio backend and a TypeScript frontend accommodating local or containerized deployment.
- **Large language model adapter:** The LLM is used as a generator and editor for PlantUML. It is invoked for initial draft generation and for subsequent edits that incorporate user feedback. UMLBot is designed to accommodate any OpenAI-compatible endpoints configured through environment variables, including API base URL, API key, and model name.
- **Prompt and context orchestration:** UMLBot assembles prompts from the selected diagram type, user request, current PlantUML artifact, and available session context. During revision, the existing PlantUML is provided back to the model so the diagram is being edited rather than generated independently.
- **PlantUML rendering:** UMLBot uses PlantUML-compatible specifications as the intermediate representation of rendered diagrams. The recommended deployment uses a local PlantUML render server started through Docker Compose, although hosted PlantUML rendering can also be configured. This design separates LLM generation from deterministic diagram rendering and allows the PlantUML source to be inspected, revised, stored, and regenerated.
- **Deployment:** UMLBot can be run locally using Docker Compose or by running the backend and frontend services directly. The full-stack Docker Compose workflow starts the PlantUML service, backend, and frontend, with the frontend available at localhost:3000. The LLM endpoint remains externally configured: it may be a commercial API, an institutional endpoint, or another OpenAI-compatible service. Thus, privacy, latency, cost, and model performance depend on the configured LLM provider.
- **Error handling and repair:** UMLBot captures errors related to LLM calls, prompt construction, PlantUML rendering, and network interactions. Errors are surfaced to the user through the interface, and when PlantUML fails to render, the system can attempt an LLM-assisted repair.

2.2. Software functionalities

UMLBot produces both editable PlantUML source and rendered diagram outputs. The PlantUML source is the primary maintainable artifact: it can be inspected, revised, stored alongside source code, and tracked in version control. This allows diagrams to evolve with the software rather than remaining static images or one-time documentation outputs. Rendered diagrams can be shared with collaborators, while the underlying PlantUML remains available for review, correction, regeneration, and extension.

2.3. Prompt construction and conversational context

UMLBot is designed to accept unstructured natural-language requests, but output quality generally improves when users provide specific context describing the intended diagram type, major actors or components, expected relationships, constraints, and/or relevant code

² - <https://plantuml.com>

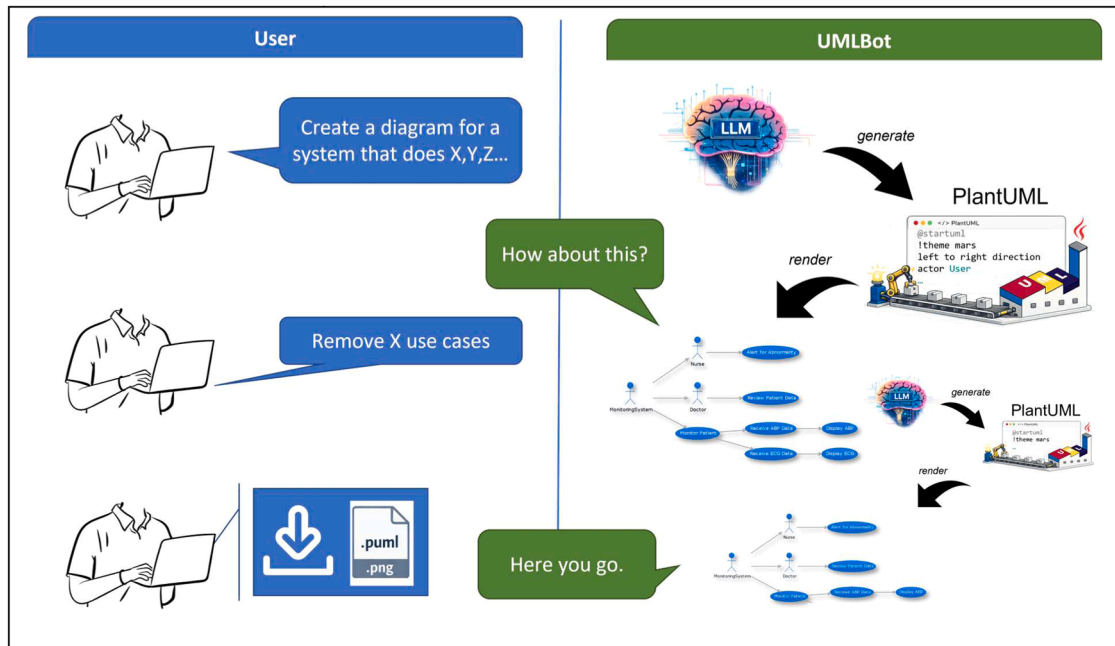


Fig. 1. Pictorial representation of the UMLBot workflow. Users describe their system of interest and send that description to UMLBot who creates and renders UML based on the interpretation of that system by an LLM. Users can edit diagrams with natural language.

excerpts. Internally, UMLBot constructs model requests using the selected diagram type, the current user message, and any available diagram context. For revision requests, the current PlantUML source can be included so the model modifies the existing artifact rather than producing a new one from scratch.

Context management is necessarily constrained by the configured LLM endpoint. Longer conversations, larger code excerpts, and more complex PlantUML artifacts increase token usage and may affect latency, cost, quality [20]. UMLBot therefore exposes the generated PlantUML to the user and treats it as the persistent artifact for review and revision, rather than relying solely on hidden conversational state. Different OpenAI-compatible models vary in their ability to follow PlantUML-specific instructions, preserve context, and repair rendering errors.

3. Illustrative examples

During early internal use, UMLBot was used to generate draft diagrams across multiple projects. This experience motivated the examples below, but should not be interpreted as a formal evaluation of diagram correctness, usability, or generalizability. The examples are intended to illustrate the workflow supported by the software: a user provides a natural-language description or code excerpt, selects a diagram type, UMLBot generates PlantUML, the application renders the diagram, and the user can iteratively request revisions, as highlighted in Fig. 1. These diagrams should be interpreted as reviewable artifacts rather than pre-validated representations of the system; domain experts still need to inspect whether components, relationships, protocols, and failure modes are complete and correct for the application in question.

In an effort toward reproducibility, the examples include the input prompt or code excerpt (see appendix), generated PlantUML source,

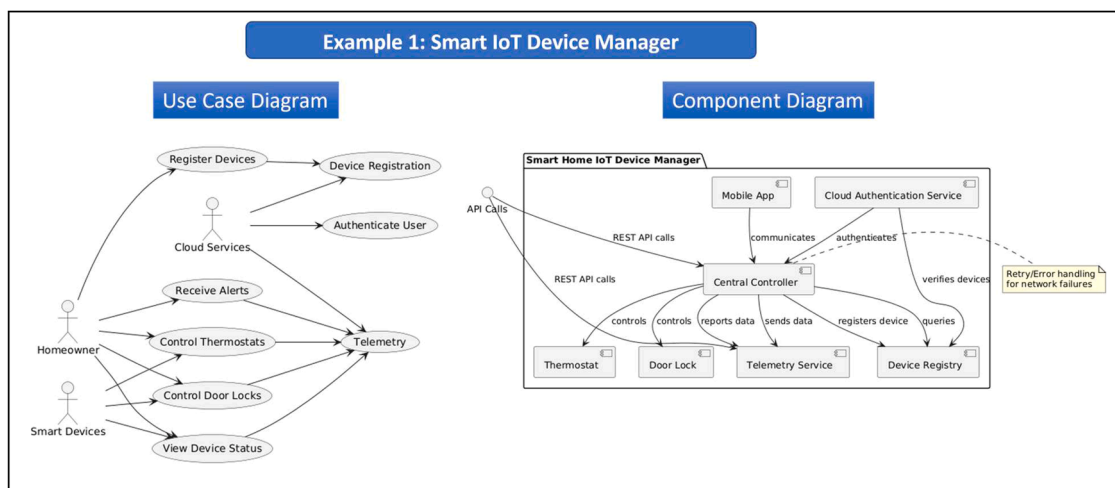


Fig. 2. Draft UML diagrams for a smart IoT device management system. The Use Case diagram depicts actors, the homeowner, cloud services, and smart devices and indicates related tasks, including controlling smart devices. The component diagram depicts the device manager with components orchestrated by a central controller. Additional details regarding prompt inputs and PlantUML text are available in the supplemental material.

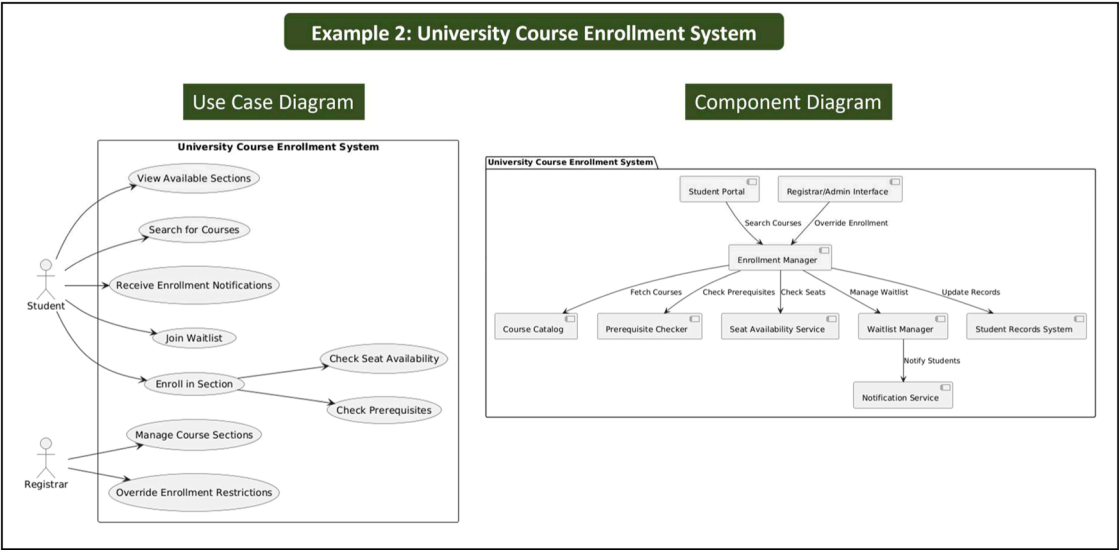


Fig. 3. Draft UML diagrams for a University course enrollment system. The use case diagram depicts actors, including the student and registrar, and indicates related tasks like enrolling in classes and managing course selection. The component diagram depicts the Enrollment Manger coordinating aspects like the student access portal and the student records system. Additional details regarding prompt inputs and PlantUML text are available in the supplemental material.

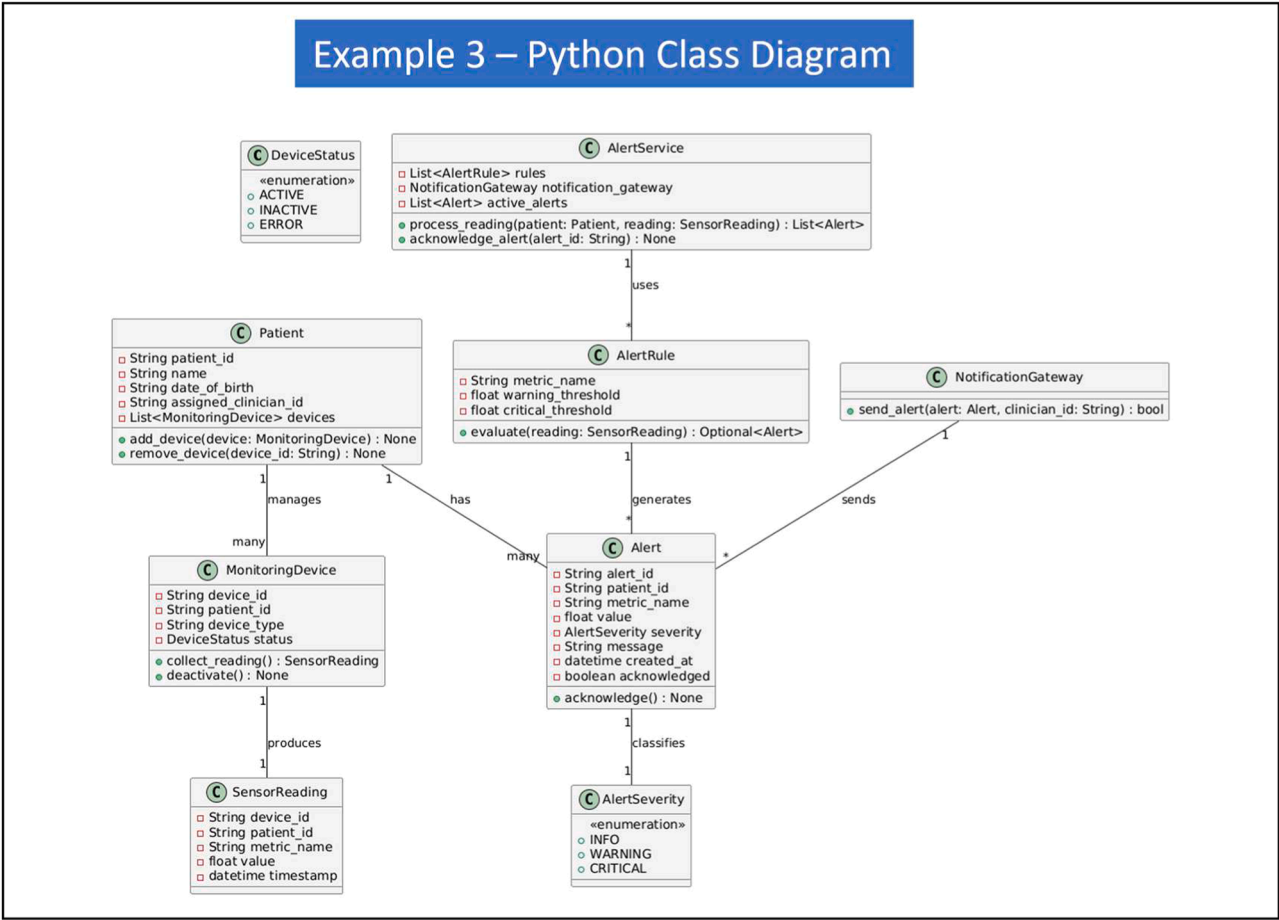


Fig. 4. Class diagram generated directly from Python code for a patient monitoring demonstration. This un-edited initial response demonstrates capture of classes, methods, attributes, and relationships, with the latter being the least reliably identified.

rendered diagram, model configuration, and the number of conversational turns used to obtain the shown diagram. However, it is worth reminding the reader that, even at zero temperature, LLM output is non-deterministic [21]:

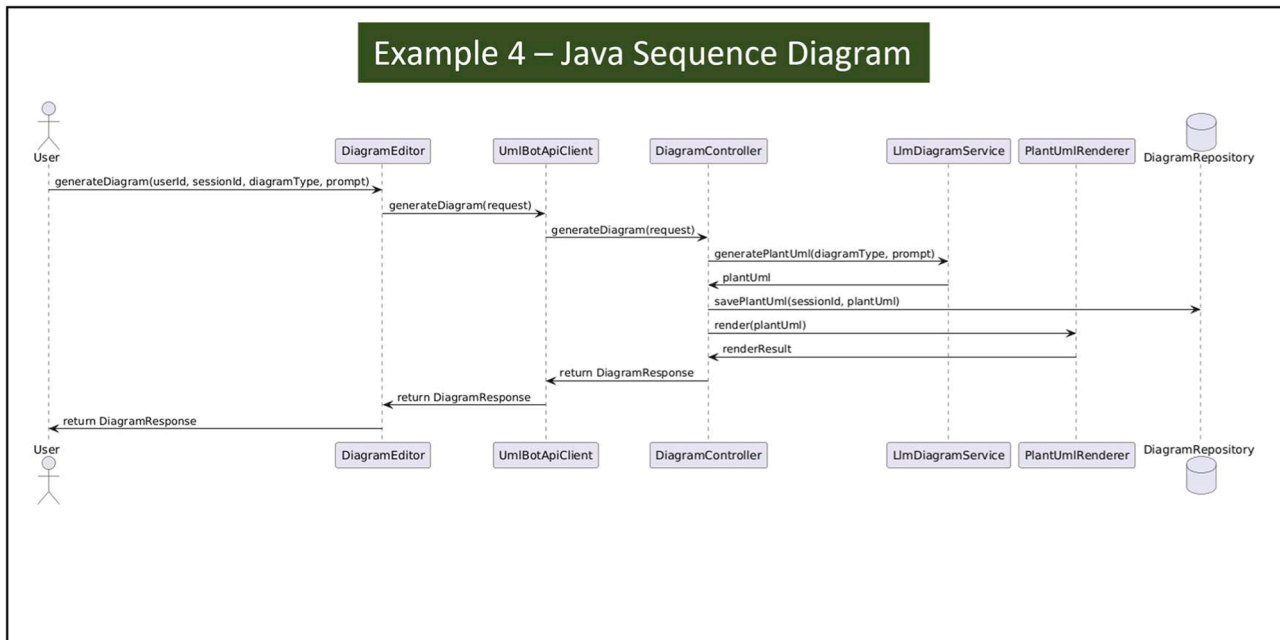


Fig. 5. Sequence diagram derived from TypeScript code for code representing the UML diagramming system described by the paper. This particular PlantUML and corresponding diagram render was generated in 3 s.

Table 1

AI Model parameters used for generating the UML diagrams in Figs 2–5.

Parameter	Description	Value
Model name	The name of the model used	Gpt-4o-mini
Date Accessed	The date the model was used	May 5, 2026
Model Type	Model compatible endpoint	Azure
Temperature	LLM Sampling temperature (lower more deterministic)	OpenAI 0.1

- **Example 1 – Smart Home IoT Device Manager:** UMLBot was used to create draft UML diagrams representing a smart home IoT device management system (Fig. 2.) The system includes a central controller coordinating devices such as thermostats and door locks within a local network, interfacing with cloud services for telemetry, device registration, and authentication. The generated diagrams illustrate how UMLBot can produce initial use case and component diagram drafts from a textual scenario.
- **Example 2 – University Enrollment System:** UMLBot was used to generate draft UML representations for a university course enrollment system (Fig. 3.) The scenario includes students searching for courses, enrolling in available sections, prerequisite checks, waitlist management, registrar actions, and notification services. The generated diagrams illustrate actors, system components, and selected interactions that can serve as a starting point for discussion among key stakeholders. As with the first example, the diagrams are intended to demonstrate UMLBot's drafting and revision workflow, not to establish semantic correctness.

Table 2

Characteristics of the example cases presented.

Example	Input Type	Diagram Type	Model	Turns	Time to (re)-render (s)	Cost
Smart home IoT	Natural Language	Use case	Gpt-4o-mini	2	3.1(2.7)	\$0.000185
		Component		1	3.6	\$0.000227
University Enrollment	Natural Language	Use case	Gpt-4o-mini	1	4.1	\$0.000197
		Component		1	2.5	\$0.000212
Python code	Code excerpt	Class	Gpt-4o-mini	1	4.6	\$0.000468
TypeScript code	Code excerpt	Sequence	Gpt-4o-mini	1	3.0	\$0.000372

- **Example 3 – Python Excerpt to Class Diagram:** To illustrate code-informed diagram drafting, UMLBot was provided with a Python code excerpt containing class definitions with various methods and attributes for a patient monitoring and alerting system (Fig. 4.) This example demonstrates that UMLBot can use code excerpts as context for draft UML generation, but it should not be interpreted as evidence of complete or general-purpose reverse engineering. The generated diagram remains a draft artifact requiring review. Generally, the LLM performs well on adding classes, methods and definitions, but is less reliable at capturing all relationships.
- **Example 4 – TypeScript/JavaScript code excerpt to sequence diagram:** UMLBot was provided with a short (250 lines) TypeScript/JavaScript code excerpt representing portions of UMLBot itself (Fig. 5.) The user requested a sequence diagram showing the order of operations for the API call illustrating how code excerpts can supplement natural-language descriptions during UML drafting.

As recommended by prior work, the LLM parameters used to create the PlantUML and corresponding diagrams are described in Table 1 [21]. Additional characteristics of the four examples are demonstrated in Table 2 and complete prompts etc. are in the appendix.

3.1. Practical considerations

In practice, useful outputs were most often obtained when users provide a specific diagram type, named the major actors or components, and described expected relationships explicitly. Common failure modes include omission of relevant components, inclusion of plausible but

unsupported relationships, PlantUML syntax errors and over-complicated diagrams that required simplification. The chat-based revision loop and exposed PlantUML source were useful in correcting these issues, but human review and critical evaluation remains an integral step.

4. Impact

UMLBot supports development of novel research and engineering workflows that rely on design communication, particularly for groups with constrained resources that struggle with the overhead of creating and maintaining diagrams. It is well suited to support interdisciplinary teams that are developing novel software, using diagrams for cognitive aids. By lowering the cost of producing and revising UML artifacts, this tool makes it more realistic to treat diagrams as evolving documentation rather than one-off deliverables.

1. **Faster iteration:** Teams can explore alternative designs quickly by editing PlantUML through natural-language requests and re-rendering immediately, which helps identify architectural issues earlier in a project.
2. **Improved (though not guaranteed) reproducibility and auditability:** Storing PlantUML alongside code enables regeneration of diagrams from a tracked source, potentially reducing documentation drift and supporting settings where traceability matters (e.g., clinical software).
3. **Enhanced collaboration:** Visual artifacts allow engineers, clinicians, and other domain experts to discuss system behavior and boundaries without requiring all participants to navigate the full codebase.

The UMLBot software complements existing documentation and reverse-engineering approaches by starting from conversational descriptions and making errors and assumptions visible to the user. Because outputs are LLM-generated drafts, they should be reviewed by interested experts like any other design artifact. The explicit PlantUML source and rendering diagnostics are intended to make that review practical and expedient.

4.1. Limitations and scope

UMLBot facilitates a workflow of creating and revising draft UML artifacts, not as a system that guarantees semantically correct diagrams. The quality of generated PlantUML depends on the underlying language model, the specificity of the user request, the selected diagram type, the complexity of the target system, and the amount and quality of user provided context. As with other LLM-assisted software engineering tools, UMLBot may return invalid PlantUML syntax (although we check for this), omit relevant components, introduce unsupported relationships, confabulate unintended elements, or produce syntactically valid PlantUML that does not accurately represent the intended system. For this reason, generated diagrams should be reviewed by domain experts in the same way as other design and documentation artifacts.

The tool is also dependent on the configured LLM. UMLBot is designed to work with OpenAI-compatible endpoints, but different models vary substantially in output quality, latency, cost, context-window size, and ability to follow PlantUML-specific instructions. The examples in this manuscript therefore should not be interpreted as evidence that all compatible models will behave similarly. However, it is worth noting, that this software is capable of leveraging a fine-tuned UML AI model that specializes in systems thinking or integrating with agentic layering to improve prompt effectiveness or automate iterations.

UMLBot can be run as a Python application, and the user interface, PlantUML source, and rendering workflow can be maintained on local or institutional infrastructure. However, unless a locally hosted or institutionally approved model endpoint is configured, user prompts and contextual information may be transmitted to an external LLM service.

This may raise privacy, governance, or compliance concerns in sensitive settings such as clinical software development. Additionally, because UMLBot accepts natural-language and code-context inputs, it may also be susceptible to prompt-injection-style instructions or dependency hallucinations common to LLM-powered software engineering tools; users should review generated PlantUML and avoid submitting sensitive or untrusted content to external model endpoints [22].

UMLBot's chat-based iterative refinement loop is intended to make errors and assumptions easier to identify and correct, and is intended to for human validation and revision. The amount of effort required to validate a diagram will vary depending on the domain, diagram complexity, and intended use. For example, a diagram used for early design discussion may require less formal review than one used to support regulated documentation or implementation planning. Similarly, the number of interactions required to reach an acceptable diagram can vary from a single prompt to multiple rounds of correction.

This manuscript does not provide a systematic quantitative evaluation of UMLBot across models, programming languages, domains, diagram types, or baseline tools. Such an evaluation would require fixed prompt suites, expert correctness ratings, and model-specific measurements of render success, semantic quality, latency, cost, and user editing effort. These analyses are important future work. The current contribution is the release of an open-source, revision-centric workflow that combines LLM-assisted PlantUML generation with automated diagram rendering with integrated error checking and reporting and supporting version-controllable diagram artifacts.

5. Conclusions

We present a new open-source software; a practical LLM-assisted workflow that converts conversational design descriptions into editable UML diagrams backed by PlantUML source. The combination of natural language-based rendering, iterative revision, and transparent error reporting reduces the friction of creating and maintaining diagrams. Future work will focus on broader diagram coverage, more systematic evaluation across domains, and usability studies that quantify how the tool affects documentation quality and project development processes.

Generative AI disclosure

Generative AI was used to support drafting initial components of this manuscript. The authors have reviewed/revised all text, checked all references and assume responsibility for the contents presented here.

CRediT authorship contribution statement

Ryan C. Godwin: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Project administration, Methodology, Investigation, Conceptualization. **Ryan L. Melvin:** Writing – review & editing, Validation, Supervision, Software, Resources, Methodology.

Declaration of competing interest

Ryan Godwin co-owns multiple LLCs (Lotz, Lacasse, Godwin, LLC - a control system software company and Blaze Pascal – a consulting firm) neither of which have any interests or connections to this work.

The authors declare no conflicts of interest with the dissemination of this work, financial or otherwise.

Acknowledgements

RG would like to acknowledge Paul J. Lotz for teaching him UML and the basics of systems design all while emphasizing the importance of human intelligence with deliberate and thorough designs.

Supplementary materials

Supplementary material associated with this article can be found, in the online version, at [doi:10.1016/j.softx.2026.102789](https://doi.org/10.1016/j.softx.2026.102789).

References

- [1] Marín B, Giachetti G, Pastor O, et al. Model-driven engineering of information systems principles, techniques, and practice 2014:111–39. <https://doi.org/10.1201/b17480-5>.
- [2] Núñez-Gaona MA, Marcelín-Jiménez R, Gutiérrez-Martínez J, et al. A dependable massive storage service for medical imaging. *J Digit Imaging* 2018;31:628–39. <https://doi.org/10.1007/s10278-018-0091-x>.
- [3] Gozali E, Safdari R, Rahimi B, et al. Designing and evaluating the children's developmental motor disorders system: an experience from a developing country. *BMC Me01d Inf Decis Mak* 2023;23:123. <https://doi.org/10.1186/s12911-023-02223-2>.
- [4] Mohanty SK, Mistry AT, Amin W, et al. The development and deployment of common data elements for tissue banks for translational research in cancer – an emerging standard based approach for the Mesothelioma Virtual Tissue Bank. *BMC Cancer* 2008;8:91. <https://doi.org/10.1186/1471-2407-8-91>.
- [5] LeBozec C, Jaulent MC, Zapletal E, et al. Unified modeling language and design of a case-based retrieval system in medical imaging. *Proc AMIA Symp* 1998:887–91.
- [6] Chen J, Wang B, Gu D, et al. Requirements analysis of real-time systems by Rational-Rose UML. In: 2009 Second Int Symp Knowl Acquis Model. 1; 2009. p. 324–7. <https://doi.org/10.1109/kam.2009.204>.
- [7] Li X, Li X, Yang A. Design and implement on integrated finance administration system. In: 2009 Second Int Symp Knowl Acquis Model. 3; 2009. p. 395–8. <https://doi.org/10.1109/kam.2009.103>.
- [8] Esiefarienrhe BM, Moemi TJ. UML design of Business Intelligence system for small-scale enterprises. *J Inf Syst Inf* 2024;6:495–513. <https://doi.org/10.51519/journalisi.v6i1.672>.
- [9] Toval A, Aleman JLF. Improving system reliability via rigorous software modeling: the UML case * granted by the CICYT (Science and Technology Joint Committee) Spanish Ministry of Education and Ministry of Industry, project MENHIR TIC97–0593-C05–02 OM, and Spanish Ministry of Science and Technology, project SIRENrm TIC 2000–1673-C06–02. In: 2001 IEEE Aerosp Conf Proc (Cat 01TH8542). 6; 2001. p. 1–12. <https://doi.org/10.1109/aero.2001.931311>.
- [10] Qu M, Wu X, Tao Y, et al. Path generation algorithm for UML graphic modeling of aerospace test software. In: IOP Conf Ser: Mater Sci Eng. 322; 2018, 062019. <https://doi.org/10.1088/1757-899x/322/6/062019>.
- [11] Briand LC, Labiche Y, Yue T. Automated traceability analysis for UML model refinements. *Inf Softw Technol* 2009;51:512–27. <https://doi.org/10.1016/j.infsof.2008.06.002>.
- [12] Chaudron MRV, Heijstek W, Nugroho A. How effective is UML modeling? *Softw Syst Model* 2012;11:571–80. <https://doi.org/10.1007/s10270-012-0278-4>.
- [13] Wan B, Wei Z, Wang S, et al. Structure-guided function-level code generation with LLMs via UML activity diagrams. *Neurocomputing* 2026;669:132502. <https://doi.org/10.1016/j.neucom.2025.132502>.
- [14] Liu X, Liu Y, Zhuang Y, et al. UCD-LLM: A use case diagram requirement modeling multi-agent framework with large language model. *Inf Softw Technol* 2026;190: 107955. <https://doi.org/10.1016/j.infsof.2025.107955>.
- [15] Nair RP, Thushara MG, Sugumaran V. Fine-tuned LLMs versus rule-based NLP for UML diagram generation: an educational evaluation. *IEEE Access* 2025;13: 211605–19. <https://doi.org/10.1109/access.2025.3638372>.
- [16] Calamo M, Mecella M, Snoeck M. Enterprise, business-process and information systems modeling, 26th International Conference, BPMDS 2025, and 30th International Conference, EMMSAD 2025, Vienna, Austria, Proceedings 2025: 211–26. https://doi.org/10.1007/978-3-031-95397-2_13.
- [17] Cohen-Adad J, Paul P, Morandi X, et al. Knowledge modeling in image-guided neurosurgery: application in understanding intraoperative brain shift. *Me01d Imaging* 2006 2006. <https://doi.org/10.1117/12.655752>. 61412E-61412E –8.
- [18] Ferrante S, Bonacina S, Pincioli F. Modeling stroke rehabilitation processes using the Unified Modeling Language (UML). *Comput Biol Med* 2013;43:1390–401. <https://doi.org/10.1016/j.combiomed.2013.07.012>.
- [19] Godwin RC, Melvin RL. Toward efficient data science: a comprehensive MLOps template for collaborative code development and automation. *SoftwareX* 2024;26: 101723. <https://doi.org/10.1016/j.softx.2024.101723>.
- [20] Klang E, Apakama D, Abbott EE, et al. A strategy for cost-effective large language model use at health system-scale. *Npj Digit Med* 2024;7:320. <https://doi.org/10.1038/s41746-024-01315-1>.
- [21] Godwin RC, Tung A, Berkowitz DE, et al. Transforming physiology and healthcare through foundation models. *Physiology* 2025;40:233–46. <https://doi.org/10.1152/physiol.00048.2024>.
- [22] AlSobeh A, Gwarzo Z, Shatnawi A. ShadowPlay: engineering defenses against role-based prompt injection and dependency hallucination in LLM-powered development. In: 2025 Int Conf Cybersec AI-Based Syst (Cyber-AI). 00; 2025. p. 317–25. <https://doi.org/10.1109/cyber-ai66431.2025.11233258>.